

UNDesparche — Referencia de API: Módulo users

Estado actual: solo el módulo `users` está disponible. Los módulos `events` e `inventory` están en desarrollo.

Base URL (desarrollo): `http://127.0.0.1:8000`

Autenticación

Todos los endpoints protegidos requieren un **ID Token de Firebase** en el header de cada request. Este token lo obtiene el frontend después de que el usuario inicia sesión con Google a través del SDK de Firebase.

```
Authorization: Bearer <firebase_id_token>
```

Importante:

- El token debe pertenecer a una cuenta con dominio `@unal.edu.co`. Cualquier otro dominio será rechazado.
 - El backend **no emite tokens** ni maneja sesiones. Firebase es el único emisor de tokens válidos.
 - El token de Firebase expira. El frontend es responsable de refrescarlo usando `getIdToken(true)` del SDK de Firebase antes de mandarlo.
 - Si un endpoint está marcado como  **público**, puede llamarse sin el header `Authorization`.
-

Comportamiento al primer login

No existe un endpoint de "registro" separado. El backend **crea automáticamente** el usuario la primera vez que llega un token válido de `@unal.edu.co`. Las siguientes veces, simplemente lo reconoce.

Flujo recomendado en el frontend:

1. Usuario hace clic en "Continuar con Google"
 2. Firebase hace el OAuth y devuelve un ID Token
 3. Frontend llama a `GET /users/me/` con ese token
 4. Si la respuesta incluye `"is_new": true` → mostrar pantalla de completar perfil
 5. Si `"is_new": false` → redirigir según el rol del usuario
-

Endpoints disponibles

GET /users/me/

Devuelve el perfil del usuario autenticado. Es el endpoint principal que el frontend debe llamar justo después de obtener el token de Firebase para saber quién es el usuario y qué puede hacer.

Request

```
GET /users/me/  
Authorization: Bearer <firebase_id_token>
```

Response exitosa — 200 OK

```
{  
  "email": "estudiante@unal.edu.co",  
  "name": "Víctor",  
  "faculty": "ING",  
  "status": "ACT",  
  "roles": ["Miembro de la Comunidad"],  
  "is_new": false  
}
```

Campos de la respuesta

Campo	Tipo	Descripción
email	string	Correo institucional del usuario
name	string	Nombre (puede estar vacío si es primer login)
faculty	string	Código de facultad (ver tabla de valores válidos abajo). Vacío si es nuevo.
status	string	Estado de la cuenta (ver tabla de valores válidos abajo)
roles	array de string	Lista de roles asignados por el administrador del sistema
is_new	boolean	true si el usuario aún no tiene facultad asignada (primer login)

Valores válidos — faculty

Código	Facultad
ING	Ingeniería
ART	Artes
CIE	Ciencias
MED	Medicina
DER	Derecho, Ciencias Políticas y Sociales

△ Esta lista se ampliará conforme se agreguen las demás facultades de la UNAL al modelo.

Valores válidos — status

Código	Significado
ACT	Activo — acceso completo a eventos e implementos
SAN	Sancionado — puede ver eventos pero no puede reservar implementos

Un usuario con cuenta desactivada (`is_active: false` internamente) simplemente recibirá un error de autenticación y nunca llegará a ver su perfil.

Valores válidos — roles

Rol	Descripción
Miembro de la Comunidad	Usuario estándar — puede consultar y reservar
Administrador de Eventos	Puede crear, editar y cancelar eventos
Administrador de Implementos	Puede gestionar el inventario de su facultad
Administrador del Sistema	Acceso total — gestión de usuarios, eventos e implementos

Un usuario puede tener más de un rol simultáneamente (el campo `roles` es un array).
Un usuario recién creado no tiene ningún rol asignado hasta que el Administrador del Sistema se lo asigne.

Códigos de error

Todos los errores siguen esta estructura:

```
{  
  "detail": "Mensaje descriptivo del error."  
}
```

Código HTTP	Cuándo ocurre
403	No se mandó el header <code>Authorization</code> , el token es inválido o está expirado
403	El correo del token no pertenece al dominio <code>@unal.edu.co</code>
403	La cuenta del usuario está desactivada (<code>is_active: false</code>)
405	Se usó un método HTTP no permitido en el endpoint (ej: POST a <code>/users/me/</code>)

Código HTTP	Cuándo ocurre
500	Error interno del servidor — reportar al equipo de backend

Ejemplo de integración en React

```
import { getAuth, signInWithPopup, GoogleAuthProvider } from "firebase/auth";

const auth = getAuth();
const provider = new GoogleAuthProvider();

async function loginConGoogle() {
  const result = await signInWithPopup(auth, provider);
  const idToken = await result.user.getIdToken();

  const response = await fetch("http://127.0.0.1:8000/users/me/", {
    headers: {
      Authorization: `Bearer ${idToken}`,
    },
  });

  if (!response.ok) {
    // 403: token inválido, dominio incorrecto, o cuenta desactivada
    throw new Error("No autorizado");
  }

  const user = await response.json();

  if (user.is_new) {
    // redirigir a pantalla de completar perfil
  } else {
    // redirigir según user.roles[0]
  }
}
```

Lo que viene próximamente

Los siguientes módulos están en desarrollo y se documentarán cuando estén disponibles:

- PATCH /users/me/ — actualizar facultad y nombre en el primer login
- GET /events/ — listado de eventos (público)
- GET /inventory/ — catálogo de implementos (requiere autenticación)